

Interpretable and Verifiable Hardware Generation with LLM-Driven Stepwise Refinement

You Li

you.li@utexas.edu

The University of Texas at Austin
USA

Samuel Mandell

shm923@my.utexas.edu

The University of Texas at Austin
USA

David Z. Pan

dpan@utexas.edu

The University of Texas at Austin
USA

Abstract

Large language models (LLMs) have achieved remarkable success in software development. However, they are susceptible to hallucinations, meaning that they can introduce subtle semantic and logical errors. Due to the high stakes in chip design and manufacturing, hardware engineers are still reluctant to rely on LLMs for register-transfer level (RTL) generation. In this paper, we propose a hardware generation framework that combines the creativity and broad knowledge of LLMs with the explainability and mathematical rigor of formal methods. Specifically, we devise a set of transformation rules that cover various design decisions and hardware features. By iteratively applying these rules, an LLM agent can convert a design specification into an RTL program with guaranteed correctness. Experimental results demonstrate the effectiveness and efficiency of the framework.

1 Introduction

Recent advances in LLMs have reshaped the landscape of programming. Starting from natural language descriptions, these models can directly synthesize executable programs for a wide range of software tasks. However, generating functionally correct RTL designs remains a significant challenge. First, most of the production-level hardware designs are proprietary. The scarcity of high-quality training data has limited the effectiveness of LLMs in the hardware domain. Second, even a single functional discrepancy in the hardware can lead to catastrophic consequences. In contrast, LLMs are prone to subtle logical and functional errors, especially when dealing with time, concurrency, and signal dependencies. Third, hardware comprises concurrent components synchronized by a clock signal. There is a fundamental mismatch between this parallel nature of hardware and the sequential generation pattern of LLMs. Fourth, existing methods generate the entire hardware design in a single pass (Section 2). The lack of intermediate steps poses challenges for performance optimization, debugging, and context management.

For these reasons, the hardware industry is still reluctant to depend on LLMs for RTL design tasks.

Formal program construction is a classical method for deriving programs with correctness guarantees [6, 16]. Given a set of coarse-grained design requirements, it incrementally constructs an executable program and formally verifies correctness at each step. Nevertheless, this approach relies heavily on human experts for both design decisions and program transformations. As a result, it has seen limited adoption in practice for developing complex reactive systems, which include integrated circuits.

In this paper, we propose an automated RTL development framework that integrates the mathematical rigor of formal program construction with the generative capabilities of LLMs. In the first stage, the natural language descriptions from the user are translated into a formal hardware specification. In the second stage, the abstract specification is refined into concrete code through a sequence of steps. At each step, an LLM is invoked to select a transformation rule from a predefined suite and then applies it to the current version. Transformation rules serve as the guardrails for LLM-guided development: if the application condition is satisfied, the resulting version is guaranteed to be a valid implementation of the previous version. Each rule contains a design decision in plain text and an algebraic transformation procedure in formal language. The stepwise development strategy and the decoupling of planning from actions enable more effective context management for LLMs. The design process is successful when the specification is completely replaced by concrete code. However, it is still possible that the development reaches a dead end, *i.e.*, it is infeasible to refine further from the current version. In this scenario, the LLM can revert the last few steps and explore a new direction. In the third stage, the generated code is converted to a synthesizable RTL program. Overall, our framework leverages the complementary strengths of formal methods and LLMs to address the hallucination and monolithic generation problems in existing methods. It also showcases the potential to build a data flywheel with verifiable rewards for hardware generation. The main contributions of this paper are summarized as follows:

Authors' Contact Information: You Li, you.li@utexas.edu, The University of Texas at Austin, USA; Samuel Mandell, shm923@my.utexas.edu, The University of Texas at Austin, USA; David Z. Pan, dpan@utexas.edu, The University of Texas at Austin, USA.

- A general and practical formalism for hardware modules. Design requirements in human language can be automatically translated into formal specifications in this representation.
- A set of LLM-compatible refinement rules for hardware design. A sound RTL implementation of the original specification can be derived by iteratively applying these rules.
- An agentic framework to generate correct-by-construction RTL programs from design requirements. It can autonomously make design decisions, apply program transformations, verify correctness, record design steps, and recover from failures.
- A thorough evaluation of the proposed framework on the VerilogEval benchmark suite. It can consistently produce reliable RTL implementations from design specifications.

2 Background and Problem Formulation

RTL Generation with LLM Agents. Agentic systems accomplish complex tasks by coordinating multiple components, such as planning, reasoning, and tool use. The main techniques employed by RTL generation agents are reflection, memory, and grounding [38]. Reflection involves refining the design in multiple steps prior to code generation [7, 47–49]. Memory-related techniques rely on in-context learning, vector databases, or knowledge graphs to supply relevant background knowledge [19, 43]. Grounding refers to connecting LLM decisions with reliable structures or external verifiers [42]. For example, VerilogCoder builds an abstract syntax tree (AST) for an RTL program to localize errors in waveforms [21]. VeriAssist utilizes execution feedback to iteratively detect and fix errors [23]. RTL++ extracts a control-data flow graph from an RTL program, allowing LLMs to deeply understand its internal behavior [2]. Meanwhile, post training techniques such as reinforcement learning with verifiable rewards (RLVR) can enhance model performance on well-defined tasks. VeriReason and RTLSeek employ reinforcement learning (RL) to improve the reasoning capabilities of LLMs [12, 44]. They combine test results from executable RTL programs with additional heuristics to construct reward models for RL training. Despite their advantages, these approaches remain monolithic in nature. The absence of structured intermediate steps and interpretable design decisions increases the verification burden, limits effective reasoning, and impedes integration with downstream design tools.

RTL Generation through High-Level Synthesis. High-level synthesis (HLS) can automatically translate an abstract behavioral model into a functionally equivalent RTL implementation [14]. It cannot be applied to an incomplete model that captures design intents and requirements. Recent work has explored how to derive a concrete behavioral model

for HLS with techniques such as refactoring and enrichment [13, 47]. However, introducing an additional level of abstraction is undesirable, as it restricts control over hardware characteristics and increases verification complexity.

Program Refinement. Refinement is a top-down approach that progressively transforms abstract requirements into executable code. The idea of program refinement can be traced back to the work of Dijkstra [15] and Wirth [45] in the 1970s. A *refinement calculus* is a collection of sound refinement rules that serve as guardrails for program refinement [5, 32, 33]. Researchers have created refinement calculi for various domains, including general concurrent systems [8, 17], reactive systems [37], real-time systems [20], and statecharts [39, 40]. Even with interactive tools that can reduce mechanical effort [6, 18, 26, 27], program refinement still relies on human creativity to choose appropriate rules and generate valid statements. Recently, Cai *et al.* suggested using Morgan’s refinement calculus [32] to guide LLMs in program generation. However, hardware description languages have unique features, such as concurrent processes, non-blocking assignments, and wait statements, which make them fundamentally different from software programming languages.

Problem Definition. We aim to establish the foundation for automated hardware generation. Given a requirements specification expressed in natural language, tables, and figures, our solution should produce an RTL implementation that complies with all requirements. In addition, the design process should be conducted in interpretable and verifiable steps.

3 Formal Specification of Hardware Requirements

In this section, we present a formal specification method that can represent design requirements, hardware behaviors, or the functionalities of HDL program segments. In contrast to prior methods [9, 30], our method is intended to be straightforward for LLMs to comprehend and reason about. This unified representation of hardware gives rise to a relatively concise refinement calculus, which will be discussed in Section 4.

Preliminaries. We consider standard first-order logic. A *term* is a variable, a constant, or a *function* symbol applied to other terms. A *predicate* is built from terms with relational operators and evaluates to either true or false. A *formula* is built from predicates with logical operators and quantifiers. In a quantified formula $\forall x.\phi$ or $\exists x.\phi$, we say that x is a *bound* variable and ϕ is the *body*. A variable that is not bound by a quantifier is called a *free* variable. Given a variable x , a term t , and a formula ϕ , we define $\phi[x/t]$ to be the formula obtained by *substituting* each free occurrence of x by t . A *state* assigns a value to each variable. Formulas ϕ and ψ are

$\langle pred \rangle ::= \text{true} \mid \text{false} \mid \langle relational\text{-}op \rangle (\langle term\text{-}list \rangle)$	$\langle expr \rangle ::= \langle id \rangle \mid \langle op \rangle (\langle expr\text{-}list \rangle)$
$\langle form \rangle ::= \langle pred \rangle \mid \langle logical\text{-}op \rangle (\langle form\text{-}list \rangle)$ $\mid \text{forall } \langle id \rangle \bullet \langle form \rangle \mid \text{exists } \langle id \rangle \bullet \langle form \rangle$ $\mid \langle temporal\text{-}op \rangle [l, u] (\langle form\text{-}list \rangle)$ $\mid \text{reset } \langle form \rangle \mid \text{assume } \langle form \rangle \mid \text{assert } \langle form \rangle$	$\langle event \rangle ::= \text{posedge}(\text{clk}) \mid \text{change}(\ast) \mid \text{change } \langle expr \rangle$ $\langle stmt \rangle ::= \text{skip} \mid \langle stmt \rangle ; \langle stmt \rangle \mid \langle id \rangle \leq \langle expr \rangle \mid \langle id \rangle := \langle expr \rangle$ $\mid \text{if } \langle expr \rangle \text{ then } \langle stmt \rangle \text{ else } \langle stmt \rangle$ $\mid \text{wait on } \langle event\text{-}list \rangle \mid \text{procedure } (\langle id\text{-}list \rangle) \{ \langle stmt \rangle \}$
$\langle spec \rangle ::= [\text{pre} : \langle form \rangle \mid \text{dur} : \langle form \rangle \mid \text{post} : \langle form \rangle] \mid \text{env} : \langle form \rangle$	$\langle bloc \rangle ::= \text{process } [\text{out} : \langle id\text{-}list \rangle] \langle stmt \rangle \mid \langle bloc \rangle \parallel \langle bloc \rangle$

Figure 1: Syntax of the formal specification language $\mathcal{L}_{spec}$ (left) and the formal specification language $\mathcal{L}_{impl}$ (right).

equivalent, denoted as $\phi \equiv \psi$, if $\phi \Leftrightarrow \psi$ holds in all states. Formula ϕ *entails* formula ψ , denoted as $\phi \Rightarrow \psi$, if $\phi \Rightarrow \psi$ holds in all states.

We use duration-bounded temporal operators to specify hardware behavior over its entire execution. Let T denote the independent time variable, and let t denote the value of T at the current time frame. $X p$ holds if p is true at time $t + 1$. $G [l, u] p$ holds if p is true for all times between $t + l$ and $t + u$. $F [l, u] p$ holds if p is true at some time between $t + l$ and $t + u$. $p U [l, u] q$ holds if q is true at some time between $t + l$ and $t + u$, and p is true for all times before q is true. When the bounds are omitted, we assume $l = 0$ and $u = \infty$. The past time versions of X , G , F , and U are Y , H , O , and S , respectively. For instance, $Y p$ holds if p is true at time $t - 1$, and it vacuously holds when t is 0.

Formal Specification Language. A *specification statement* represents an abstract program [32]. During program refinement, it denotes a program fragment that remains to be developed. In the software domain, a specification statement has the form $w : [pre, post]$, where w is a set of variables, and pre , $post$ are predicates. The precondition pre constrains the initial state, the postcondition $post$ constrains the final state, and the frame w lists all variables that may change values during the transition. We assume that no transition occurs from an initial state that does not satisfy the precondition [17]. The specification statement describes the following task for the developer: from any initial state satisfying the precondition, the program fragment must terminate in a state satisfying the postcondition, by modifying only the frame variables.

A hardware module may execute for an extended period of time or even indefinitely. As a result, it is necessary to specify the behavior during execution [10]. Meanwhile, the correct operation of a process may depend on the environment assumptions and the behaviors of other synchronized processes. For these reasons, we introduce a *during condition* dur and an *environment condition* env into the specification statement, which leads to the following form: $w : [pre, dur, post] \mid env$. The program fragment must maintain the during condition throughout its execution, and it can assume the environment condition as the context for development. The pair of env and dur conditions is similar to the rely/guarantee approach in reactive system modeling [1, 24].

When developing a program fragment of a process body, we incorporate the environment condition into the *during condition*, which yields a simplified specification statement $w : [pre, dur, post]$. Since a loop-free process body always terminates, these two conditions jointly provide the context for development. We do not include liveness properties in the specification statements. Instead, we encode progress requirements into the LLM prompts used for development.

Hardware Description Language. We define a simple language $\mathcal{L}_{impl}$ to represent hardware implementations. A logic circuit is a network of interconnected *processes*, each triggered by the designated *events* in its sensitivity list. Once triggered, the *statements* within a process are evaluated sequentially. These include control statements, signal assignment statements, and wait statements. The sequence assignment statement, denoted as $\langle id \rangle := \langle expr \rangle$, takes effect immediately when it is evaluated. The concurrent assignment statement, denoted as $\langle id \rangle \leq \langle expr \rangle$, schedules a new value for its left-hand side signal based on the current value of its right-hand side expression. After an infinitesimal delay, the scheduled updates of all concurrent assignments within the same process take effect simultaneously. The *wait* statement suspends the execution of a process until a designated event occurs. Intuitively, its functionality is similar to that of the sensitivity list in the process declaration [3]. Extra delays in statements are not allowed because they cannot be synthesized into hardware.

The Backus-Naur Form (BNF) of the hardware description language $\mathcal{L}_{impl}$ is shown in Fig. 1(right). This language is functionally complete, as it can represent any concrete behavior specified by the formal specification language [8].

4 A Refinement Calculus for LLM-Driven Hardware Design

Program Refinement. Refinement is a top-down approach for program development. It starts with an abstract specification statement, proceeds through a series of intermediate program versions, and results in code in a programming language:

$$spec \sqsubseteq ver_1 \sqsubseteq \dots \sqsubseteq ver_n \sqsubseteq code. \quad (1)$$

Every intermediate version may be a mixture of specification statements and code. A version P is refined by a version Q ,

denoted $P \sqsubseteq Q$, if every execution that satisfies Q also satisfies P [31]. Intuitively, a refined version is more deterministic than the previous version. In that sense, refinement can be viewed as a process to progressively reducing nondeterminism.

Refinement Rules. Every $\sqsubseteq$ symbol in a refinement trace (Equation 1) represents a *refinement relation* between two consecutive versions. Rather than relying on human experts to define a new refinement relation at each step, researchers have developed the *refinement calculus*, which provides a systematic set of predefined refinement rules. The final code derived with the refinement calculus is correct by construction if all the refinement rules are sound. In this section, we introduce a refinement calculus for LLM-driven hardware design. In our approach, each refinement rule consists of three components: (i) a *design decision*, (ii) an *application condition*, and (iii) an *algebraic transformation*. The design decision is expressed in natural language to fully leverage LLMs’ capabilities, including semantic comprehension, task planning, and information retrieval. The other two components are expressed in formal language to provide mathematical guarantees of functional correctness. The application condition describes the scenarios in which it is sound to apply this rule. The transformation encapsulates a predefined refinement relation and specifies how to derive the next version. This separation of concerns facilitates more effective agent orchestration and context management in an LLM-driven autonomous system.

Design Mechanism. The problem of program refinement exhibits a tree structure once the initial specification statement and the refinement calculus are determined. In a *refinement tree*, the root node corresponds to the initial specification statement, each intermediate node is a program version, each real leaf node is a valid implementation, and each dummy leaf node is an infeasible program version. A design agent aims to find a path from the root to a real leaf node by repeating its action: selecting a fragment of the current program version and applying an appropriate refinement rule to that location. Each action corresponds to an edge in the refinement tree. During the search process, the agent may encounter an infeasible version that cannot be implemented by any code. In that case, the agent needs to revert the last few refinement steps and find another direction. Depending on user requirements, the search can be steered toward specific directions that may yield optimal results in power, performance, and area. Search efficiency can be further improved using techniques such as in-context reinforcement learning, information retrieval, and compositional refinement rules [11]. We describe our strategy in detail in Section 5.

Refinement Calculus for Hardware Development. An integrated circuit can be modeled as a network of processes

that respond to designated events on their input signals. A process is activated when an event in its sensitivity list occurs, after which every statement in the process body is evaluated. A standalone signal assignment statement can also be viewed as a process [3]. In this paper, we enforce a simple design pattern for all design problems. In the first phase, an agent structurally and functionally partitions the original specification statement into one or more concurrently running processes. For each process, the agent consolidates its behavior and extracts a specification statement representing the process body. In the second phase, an agent generates concrete statements that implement the process body. This design pattern reduces the size of the refinement tree and avoids unnecessary backtracking during the tree search. Moreover, it ensures that each process is refined individually, and thus mitigates the context dilution problem in monolithic LLM-guided approaches [22].

We provide the essential rules for process-level development and control-data flow development in Table 1 and Table 2. We display the transformations and their application conditions in a symbolic format to facilitate reproduction. In the following, we give an intuitive explanation of these rules.

The *weaken precondition* and the *strengthen postcondition* are the most fundamental rules to refine a process body. The former makes the specification statement more generally applicable, and the latter yields a more deterministic program. Analogously, *weaken environment* and *strengthen during* are the most fundamental rules to refine a continuously running process [28]. Recall that a specification statement may only modify the variables in its frame. The *contract frame* and the *expand frame* rules allow for the addition and removal of frame variables.

We define a set of rules to reduce the problem of refining a single process to the problem of refining two concurrent processes A and B . The *Parallel Composition* rule applies when A and B have no cross-references between their output signals; the *Piping Composition* rule applies when B references the output signals of A , but not vice versa; the *Bidirectional Composition* rule applies when there is mutual dependency. The during conditions of A or B must not reference the frame variables of the other process, and their conjunction must be sufficiently strong to establish the during condition of the original process. Furthermore, the during condition of the *producer* process should be reflected in the environment condition of the *consumer* process, so that each can be refined in isolation.

We create *Initialization* and *Iteration* rules to reduce the problem of refining a process to the problem of refining the process body. A process can be viewed as an infinite loop preceded by an initialization of the state variables [8]. The

Name	Transformation and Conditions
<i>Weaken Environment</i>	$ss \sqsubseteq w : [pre', dur, post] \mid env', \text{provided } env \Rightarrow env'.$
<i>Strengthen During</i>	$ss \sqsubseteq w : [pre, dur', post] \mid env, \text{provided } dur' \Rightarrow dur.$
<i>Parallel Composition</i>	$sf \sqsubseteq w_a : [pre, dur_a, false] \mid env \parallel w_b : [pre, dur_b, false] \mid env, \text{provided}$ $i) w_a \cap w_b = \emptyset, ii) w_a \notin vars(dur_b) \wedge w_b \notin vars(dur_a), iii) dur_a \wedge dur_b \Rightarrow dur.$
<i>Piping Composition</i>	$sf \sqsubseteq w_a : [pre, dur_a, false] \mid env \parallel w_b : [pre, dur_b, false] \mid env \wedge dur_a,$ $\text{provided } i) w_a \cap w_b = \emptyset, ii) w_a \in vars(dur_b) \wedge w_b \notin vars(dur_a), iii) dur_a \wedge dur_b \Rightarrow dur.$
<i>Bidirectional Composition</i>	$sf \sqsubseteq w_a : [pre, dur_a, false] \mid env \wedge dur_b \parallel w_b : [pre, dur_b, false] \mid env \wedge dur_a,$ $\text{provided } i) w_a \cap w_b = \emptyset, ii) w_a \in vars(dur_b) \wedge w_b \in vars(dur_a), iii) dur_a \wedge dur_b \Rightarrow dur.$
<i>Initialization</i>	$ss \sqsubseteq w : [pre, dur \wedge rst \Rightarrow (T = 0 \wedge w = \text{reset_state}), post] \mid env,$ $\text{provided } i) w = \text{reset_state} \Rightarrow pre, ii) T = 0 \wedge w = \text{reset_state} \Rightarrow T = 0 \wedge dur.$
<i>Iteration</i>	$sf \sqsubseteq \text{process}[\text{output} : w] \{ w : [T = t_0 \wedge inv, env \wedge dur, T = (t_0 + 1) \wedge inv] \},$ $\text{provided } T = 0 \wedge pre \Rightarrow T = 0 \wedge inv.$

Table 1: Refinement rules for process-level development. ss is the shorthand for $w : [pre, dur, post] \mid env$, and sf is the shorthand for $w : [pre, dur, false] \mid env$.

Name	Transformation and Conditions
<i>Weaken Precondition</i>	$ss \sqsubseteq w : [pre', dur, post], \text{provided } pre \Rightarrow pre'.$
<i>Strengthen Postcondition</i>	$ss \sqsubseteq w : [pre, dur', post'], \text{provided } post' \Rightarrow post.$
<i>Weaken Context</i>	$ss \sqsubseteq w : [pre, dur', post], \text{provided } i) dur \text{ contains no temporal operators, } ii) dur \Rightarrow dur'.$
<i>Expand Frame</i>	$ss \sqsubseteq w : [pre, dur, post \wedge x = x_0].$
<i>Contract Frame</i>	$w, x : [pre, dur, post] \sqsubseteq w : [pre, dur, post[x_0/x]].$
<i>Sequential Composition</i>	$w, x : [pre, dur, post] \sqsubseteq w : [pre, dur, mid]; w : [mid, dur, post], \text{provided}$ $i) w_0, x_0 \notin vars(mid), ii) pre \wedge dur \Rightarrow mid, iii) mid \wedge dur \Rightarrow post.$
<i>Blocking Assignment</i>	$w, x : [pre, dur, post] \sqsubseteq w := E, \text{provided } pre \wedge dur \Rightarrow post[w/E].$
<i>Non-Blocking Assignment</i>	$w, x : [pre, dur, post] \sqsubseteq w <== E, \text{provided } pre \wedge dur \Rightarrow post[T/(T + 1)][w/E].$
<i>Concurrent Assignment</i>	$w := E; x := F \sqsubseteq w, x := E, F[w/E], \text{provided } w \cap x = \emptyset.$
<i>Leading Assignment</i>	$w, x := E, F[w/E] \sqsubseteq w := E; x := F, \text{provided } w \cap x = \emptyset.$
<i>Following Assignment</i>	$w, x : [pre, dur, post] \sqsubseteq w, x : [pre, dur, post[w/E]]; w := E, \text{provided } pre \wedge dur \Rightarrow post[w/E].$
<i>Skip Statement</i>	$ss \sqsubseteq \text{skip}, \text{provided } pre \wedge dur \Rightarrow post.$
<i>Introduce Variable</i>	$ss \sqsubseteq \text{Var } x; w, x : [pre, dur, post].$
<i>Alternation</i>	$ss \sqsubseteq \text{if } G_i \text{ then } \{ w : [pre \wedge G_i, dur, post] \} \text{ for } i = 1 \dots n.$
<i>Procedure Assignment</i>	$w, a := E[f/a], F[f/a] \sqsubseteq P(a), \text{provided procedure } P(f) \triangleq w, f := E, F.$
<i>Procedure Specification</i>	$w, a : [pre, dur, post] \sqsubseteq P(a), \text{provided } i) \text{procedure } P(f) \triangleq w, f : [pre', true, post'],$ $ii) pre \wedge dur \Rightarrow pre'[f/a], iii) post'[f/a] \Rightarrow post.$
<i>Feasibility</i>	$ss \text{ is feasible, provided } pre \wedge dur \Rightarrow \exists w.post.$

Table 2: Refinement rules for control and data flow development. ss is the shorthand for $w : [pre, dur, post]$.

Initialization rule allows information flow from the precondition to the during condition. It represents the design decision to choose a `reset_state` that satisfies the precondition and to encode it into the during condition. The *Iteration* rule extracts the specification statement of the process body from that of the entire process. It represents the design decision to implement the given specification as a single process and to choose an *inductive invariant* for that process. Because a specification statement can change the values of the frame variables and the time variable T , we use the 0-subscripted versions to denote their values in the precondition. The inductive invariant *inv* must hold at all discrete instants of the

timeline: at the beginning of every iteration when T is set to t_0 , and at the end of every iteration when T is updated. In addition, it must be sufficiently strong to establish the during condition of the entire process. The agent has the flexibility to choose an inductive invariant that satisfies the above conditions. In practice, choosing an appropriate *inv* under these constraints is much easier than finding an inductive invariant for a hardware implementation. For example, the agent can simply strengthen the during condition to obtain an *inv*. If subsequent refinement of the process body fails, the agent can backtrack and explore a different *inv*. The environment condition, which holds throughout the process

execution, is integrated into the during condition of the process body. Such a during condition does not constrain the frame variables; instead, it provides the conditions that the process body can rely on during its execution.

We extend a set of software refinement rules [31] to the control-data flow development of a process body (Table 2). Because the during condition holds throughout the execution of the process body, it serves as the *context* when reasoning about the application conditions. Notably, the *Concurrent Assignment* and the *Leading Assignment* rules enable the sound conversion between a single concurrent assignment statement and multiple sequential assignment statements, and vice versa. The *Procedure Assignment*, and *Procedure Specification* rules allow the reuse of already refined, parametrized modules. Finally, the *Feasibility* condition checks whether the specification statement is valid and can be refined further.

5 An Agentic System for Correct-by-Construction Hardware Development

We develop an end-to-end, fully automated agentic system for hardware design based on the formal techniques discussed in this paper. Our main design considerations are simplicity and robustness. The system is intentionally kept simple so that it does not depend on external knowledge bases and can be maintained by engineers without formal verification expertise. It is designed to be robust so that it consistently follows the guardrails and delivers reliable results under varying input specifications. We choose *Lang-Graph* as our development framework because it offers fine-grained control over agent behaviors. We use *Dafny* as our programming language and verification tool. *Dafny* has built-in support for specification constructs such as preconditions (requires), postconditions (ensures), and frames (modifies). It also comes with an SMT-based static verifier to check whether a program is functionally correct with respect to the specifications. As such, we can verify the correctness of a mixed program during its construction. The general architecture of the system is shown in Fig. 2. In this figure, each rectangular node represents an LLM call, each pair of parentheses represents a tool call, and each document shape represents a block of short-term memory. The system consists of three orthogonal stages, which are elaborated as follows.

Auto-Formalization Stage. In the first stage, the agent parses a design specification expressed in natural language, flow charts, truth tables, or waveforms. The *Formalization* node first determines whether the design specification corresponding to a combinational circuit or a sequential circuit. It then extracts the functional requirements from the design specification. For a combinational circuit, the environment

condition is incorporated into the during condition, and the entire execution is finished in one clock cycle. We define dedicated constructs so that *Dafny* can support hardware-specific features. For instance, we flatten formulas with a bounded temporal operator into formulas over the time variable *T*, and define bit-shift operators as customized functions in *Dafny*. Afterwards, the *Syntax Checking* node launches *Dafny* via the command-line interface to detect syntax errors in the formal specifications, and the *Functional Judge* node invokes an LLM to identify functional discrepancies with respect to the design specification. This loop terminates when all syntactic and functional issues are resolved. The final output is a specification statement in the format described in Section 3. In practice, a design engineer can review the specification statement and make desired improvements.

Stepwise Refinement Stage. The second stage takes both the generated specification statement and the original design specification as input. Initially, the *Planning* node sketches a high-level design plan according to the non-functional requirements [36] in the design specification, along with its own design decisions. The design plan contains the following fields: a process-level model [4] that illustrates the partitioning of the system and the interconnections among the processes; port definitions that describe the signal mappings between processes; resource, scheduling, and signal path constraints on each individual process; high-level descriptions of data path and control logic for each individual process; a general design pattern for the top-down, iterative refinement strategy; and other information the LLM deems necessary. Next, the *Refinement Decision* node makes a single design decision, selects an appropriate refinement rule, and identifies a program location to apply this rule. It maintains a complete history of refinement steps, so that it can learn from previous mistakes and backtrack to a past version. Subsequently, the *Program Transformation* node updates the current program version based on the refinement rule and the program location passed from the refinement decision node. The above iterative refinement loop terminates when the *Termination Checking* node finds that all specification statements have been replaced by concrete code. The clear separation between decision, action, and verification enables effective context management for each LLM call, which in turn mitigates context dilution and reduces token consumption.

It is always difficult to verify a complex system after design completion. In that regard, we perform incremental verification alongside each design iteration. The *Condition Checking* node validates the application conditions once a refinement rule is selected. The *Feasibility Checking* node determines whether the newly generated refinement statements can be refined further. Furthermore, the *Termination Checking* node consistently verifies whether the current implementation

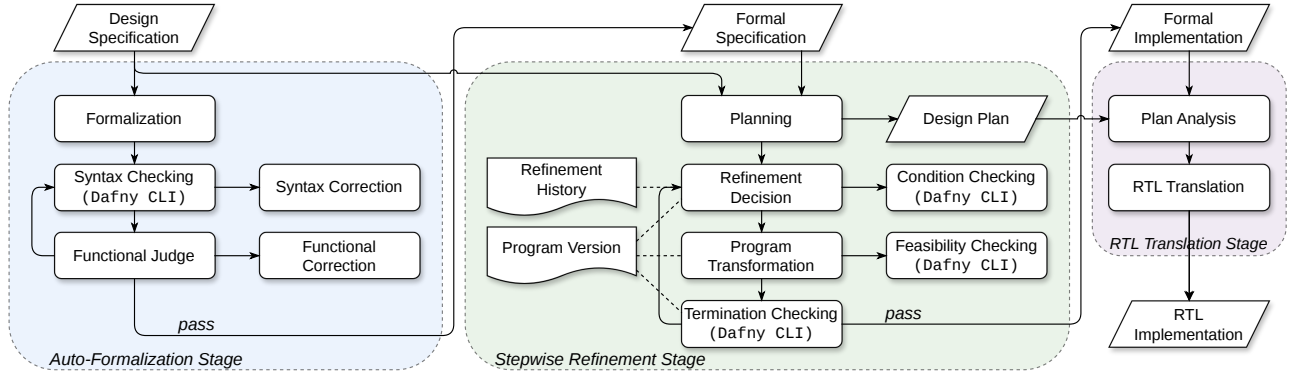


Figure 2: Architecture of the agentic system for correct-by-construction RTL generation.

of a process body satisfies the associated Dafny specifications. It leverages Dafny’s incremental verification features, such as extension, strengthening, and superimposition, if the refinement rule is compatible with them [25]. Otherwise, it inserts `assume` and `assert` statements around the program location to ensure the correctness of the refinement step. Overall, the tight integration of design and verification reduces verification complexity, enables error detection in early stages, and provides intermediate reward signals for in-context reinforcement learning.

RTL Translation Stage. The final stage receives the concrete Dafny program and the design plan from the previous stage. The *Plan Analysis* node extracts auxiliary information from the design plan, including port definitions, register definitions, and whether each process should be implemented as a sequential block, a combinational block, or an assign statement. Lastly, the *RTL Translation* node converts each process signature and statement into Verilog. The end result is a synthesizable algorithmic RTL implementation that satisfies all initial design requirements.

6 Evaluation

Experimental Setup. We implement the proposed agentic system (Fig. 2) on Dafny 4.0 [27] and LangGraph, with approximately 4,000 lines of Python code and LLM prompts. We use Claude Opus 4.6 for the *Formalization* and *Planning* nodes, and Claude Sonnet 4.6 for the rest of the nodes. We find that the Claude 4.6 family of models consistently delivers better semantic comprehension, logical reasoning, and instruction following capabilities than their counterparts. To ensure a fair comparison, we use Claude Opus 4.6 for all baseline methods.

We evaluate different RTL generation methods on *VerilogEval V2*, a benchmark suite comprising 156 hardware design problems [34]. Each benchmark consists of a natural

language description of the desired functionality, a reference Verilog implementation, and a testbench for functional verification. A generated RTL design is deemed correct if it passes all test cases and is approved by a human expert. VerilogEval is considered contaminated, since the reference implementations in the public repository may have been seen by LLMs during pretraining [35]. However, to the best of our knowledge, no previous systems have adopted a top-down, stepwise refinement approach for hardware design. In this sense, VerilogEval remains a fresh benchmark suite for our method.

We compare our method with off-the-shelf Claude Opus 4.6 and an advanced multi-agent RTL generation system called *VeriMaAS* [7]. *VeriMaAS* combines the functional verification capabilities of *Yosys* [46] with the reasoning capabilities of agent orchestration to achieve high generation quality. All verification tasks are conducted on a Windows laptop with an Intel i7-12700H CPU and 32 GB of RAM.

Experimental Results. Table 3 shows the evaluation results of the three RTL generation methods on the VerilogEval benchmarks. As we have discussed, our proposed method is at a disadvantage, because LLMs are not pretrained for hardware design with a top-down, stepwise refinement approach. We compare the results of running our method once and the other two methods ten times on each benchmark. For *VeriMaAS*, the runtime is not directly comparable, as it includes a physical design flow for performance estimation. Across all benchmarks, our method achieves the highest pass rate, yet it consumes more tokens and runtime than the other two methods. This trade-off is justified by the high correctness stakes in chip design and manufacturing.

Table 4 shows the detailed statistics of our method. It achieves a lower pass rate on sequential benchmarks due to the challenges of process-level reasoning in addition to control and data flow design. Most of the runtime and tokens are used by Stage 2, where the stepwise refinement

Method	Metrics	Results
Proposed	Pass@1 (%)	92.3 (144/156)
	Avg. Runtime (s)	221.1
	Avg. Tokens (k)	61.8
Claude Opus 4.6	Pass@10 (%)	87.2 (136/156)
	Avg. Runtime (s)	52.9
	Avg. Tokens (k)	6.9
VeriMaS [7]	Pass@10 (%)	90.0 (141/156)
	Avg. Runtime (s)	–
	Avg. Tokens (k)	17.1

Table 3: A comparison of different RTL generation methods.

Metrics	Category	Results
Pass@1 (%)	Combinational	95.1 (78/82)
	Sequential	89.2 (66/74)
Avg. Runtime (%)	Stage 1	14.0
	Stage 2	80.6
	Stage 3	5.4
Avg. Tokens (%)	Stage 1	14.6
	Stage 2	74.0
	Stage 3	11.4
Avg. Search Depth	–	7.1
Backtrack Rate (%)	–	2.6
Cache Hit Rate (%)	–	53.2

Table 4: Detailed statistics of the proposed method.

agent iteratively makes design decisions and applies program transformations. Notice that Stage 2’s proportion of runtime is only slightly higher than its proportion of token consumption. It indicates that the time spent on formal verification is significantly less than the time spent on LLM calls. We attribute the high efficiency of formal verification to the incremental verification strategy and the clear definition of application conditions. The stepwise refinement agent takes an average of 7.1 steps on the refinement tree to find a valid implementation. Moreover, it reaches a dead end in only 2.6% of the steps. These numbers suggest a high potential for deriving compositional refinement rules and training a more aggressive search agent through reinforcement learning. We leave these directions for future work.

Failure Analysis. We analyze the root causes of all failures. On benchmarks 062, 063, and 093, the design specifications contain ambiguities. Our generated implementations satisfy the given functional requirements, but deviate from the reference implementations. On benchmarks 034, 053,

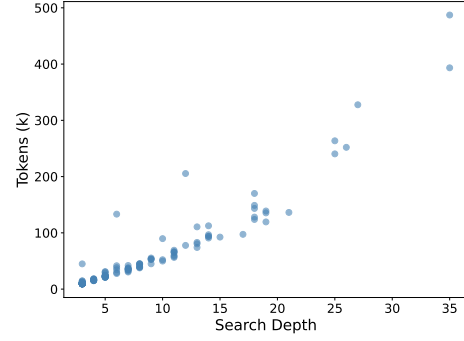


Figure 3: Number of tokens consumed in stage 2 vs. search depth on the refinement tree.

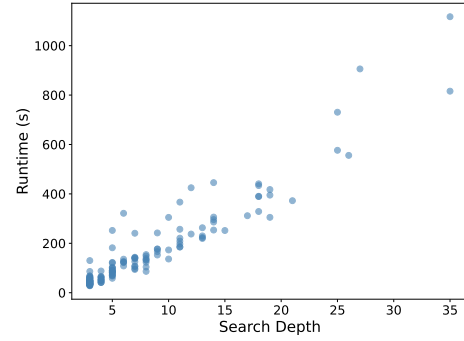


Figure 4: Total runtime of stage 2 vs. search depth on the refinement tree.

078, 099, 104, 137, 145, and 149, our generated implementations choose a different time schedule or reset state than the reference implementations, yet they still exhibit correct observable behaviors. On benchmark 070, our method selects a design strategy that violates the non-functional requirements. In summary, the proposed method can consistently deliver correct functionality, while it still depends on LLMs for other design considerations.

Scalability Analysis. As shown in Fig. 3 and Fig. 4, the token consumption and runtime of Stage 2 grow almost linearly as the number of refinement steps grows. The underlying reasons include (a) that the backtrack rate (Table 4) is nearly negligible; (b) that program transformation is applied to just a single specification statement or program fragment at a time; and (c) that the number of application conditions is constant for each refinement rule. Hence, we expect our method to scale to complex designs that require a large number of design decisions.

7 Conclusion

In this paper, we propose the first LLM-driven hardware design method based on program refinement. Our method can also be viewed as (a) an inference-time scaling technique with formal correctness guarantees, (b) an automated system for stepwise program refinement, and (c) a synthetic data generator for interpretable hardware synthesis. Based on this method, we construct an agentic system that consistently produces reliable RTL implementations from design specifications. Future directions include (a) building scaffolding for the transformation, verification, and compilation of formal hardware languages, (b) developing more general and powerful refinement rules for hardware, and (c) using reinforcement learning to improve LLMs’ capabilities in hardware refinement.

References

- [1] Martin Abadi and Leslie Lamport. 1993. Composing specifications. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 15, 1 (1993), 73–132.
- [2] Mohammad Akyash, Kimia Azar, and Hadi Kamali. 2025. Rtl++: Graph-enhanced llm for rtl code generation. In *ICLAD 2025*.
- [3] James R Armstrong and F Gail Gray. 1993. *Structured logic design with VHDL*. Prentice-Hall, Inc.
- [4] James R Armstrong and F Gail Gray. 2000. *VHDL design representation and synthesis*. Prentice Hall PTR.
- [5] Ralph-JR Back. 1988. A calculus of refinements for program derivations. *Acta Informatica* 25, 6 (1988), 593–624.
- [6] Friedrich L Bauer, Bernhard Moller, Helmuth Partsch, and Peter Pepper. 1989. Formal program construction by transformations-computer-aided, intuition-guided programming. *IEEE Transactions on Software Engineering* 15, 2 (1989), 165–180.
- [7] Amulya Bhattaram, Janani Ramamoorthy, Ranit Gupta, Diana Marculescu, and Dimitrios Stamoulis. 2025. Automated Multi-Agent Workflows for RTL Design. *NeurIPS 2025 Workshop MLForSys* (2025).
- [8] Peter T Breuer, Carlos Kloos Delgado, Andres Lopez Marin, Natividad Martinez Madrid, and Luis Sanchez Fernandez. 1997. A refinement calculus for the synthesis of verified hardware descriptions in VHDL. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 19, 4 (1997), 586–616.
- [9] Peter T Breuer, Luis Sanchez Fernandez, and Carlos Delgado Kloos. 1995. A simple denotational semantics, proof theory and a validation condition generator for unit-delay VHDL. *Formal Methods in System Design* 7, 1 (1995), 27–51.
- [10] Peter T Breuer, Natividad Martinez Madrid, Luis Sánchez, Andrés Marin, and Carlos Delgado Kloos. 1996. A formal method for specification and refinement of real-time systems. In *Proceedings of the Eighth Euromicro Workshop on Real-Time Systems*. IEEE, 200–204.
- [11] Yufan Cai, Zhe Hou, David Sanán, Xiaokun Luan, Yun Lin, Jun Sun, and Jin Song Dong. 2025. Automated program refinement: Guide and verify code large language model with refinement calculus. *POPL 2025* (2025).
- [12] Zhiteng Chao, Xinyu Zhang, Yonghao Wang, Bin Sun, Tianyun Ma, Tianmeng Yang, Jing Justin Ye, Jianan Mu, and Huawei Li. 2026. RTLSeek: Boosting the LLM-Based RTL Generation with Diversity-Oriented Reinforcement Learning. (2026).
- [13] Luca Collini, Siddharth Garg, and Ramesh Karri. 2025. C2hlsc: Leveraging large language models to bridge the software-to-hardware design gap. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 1–24.
- [14] Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees Vissers, and Zhiru Zhang. 2011. High-level synthesis for FPGAs: From prototyping to deployment. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 30, 4 (2011), 473–491.
- [15] Ole-Johan Dahl, Edsger Wybe Dijkstra, and Charles Antony Richard Hoare. 1972. *Structured programming*. Academic Press Ltd.
- [16] Edsger Dijkstra. 1976. *A discipline of programming*. Prentice-Hall, Inc.
- [17] Jürgen Dingel. 2002. A refinement calculus for shared-variable parallel and distributed programming. *Formal Aspects of Computing* 14, 2 (2002), 123–197.
- [18] Iulia Dragomir, Viorel Preoteasa, and Stavros Tripakis. 2020. The refinement calculus of reactive systems toolset. *International Journal on Software Tools for Technology Transfer* 22, 6 (2020), 689–708.
- [19] Fanghao Fan, Yingjie Xia, and Li Kuang. 2025. SecV: LLM-based Secure Verilog Generation with Clue-Guided Exploration on Hardware-CWE Knowledge Graph. In *IJCAI 2025*.
- [20] Ian J Hayes and Mark Utting. 2001. A sequential real-time refinement calculus. *Acta Informatica* 37, 6 (2001), 385–448.
- [21] Chia-Tung Ho, Haoxing Ren, and Bruce Khailany. 2025. Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool. In *AAAI 2025*.
- [22] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekeshe, Fei Jia, and Boris Ginsburg. 2024. RULER: What’s the Real Context Size of Your Long-Context Language Models?. In *COLM 2024*.
- [23] Hanxian Huang, Zhenghan Lin, Zixuan Wang, Xin Chen, Ke Ding, and Jishen Zhao. 2024. Towards llm-powered verilog rtl assistant: Self-verification and self-correction. *arXiv preprint arXiv:2406.00115* (2024).
- [24] Cliff B. Jones. 1983. Tentative steps toward a development method for interfering programs. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 5, 4 (1983), 596–619.
- [25] Jason Koenig and K Rustan M Leino. 2016. Programming language features for refinement. *arXiv preprint arXiv:1606.02022* (2016).
- [26] Peter Lammich and Andreas Lochbihler. 2019. Automatic refinement to efficient data structures: A comparison of two approaches. *Journal of Automated Reasoning* 63, 1 (2019), 53–94.
- [27] K Rustan M Leino. 2010. Dafny: An automatic program verifier for functional correctness. In *International conference on logic for programming artificial intelligence and reasoning*. Springer, 348–370.

- [28] Brendan P Mahony and Ian J Hayes. 1992. A case-study in timed refinement: A mine pump. *IEEE transactions on Software Engineering* 18, 9 (1992), 817–826.
- [29] Zohar Manna and Amir Pnueli. 1990. A hierarchy of temporal properties (invited paper, 1989). In *Proceedings of the ninth annual ACM symposium on Principles of distributed computing*. 377–410.
- [30] Patrick Meredith, Michael Katelman, José Meseguer, and Grigore Roşu. 2010. A formal executable semantics of Verilog. In *MEMOCODE 2010*.
- [31] Carroll Morgan. 1988. The specification statement. *ACM Transactions on Programming Languages and Systems* 10, 3 (1988), 403–419.
- [32] Carroll Morgan. 1990. *Programming from specifications*. Prentice-Hall, Inc.
- [33] Joseph M Morris. 1987. A theoretical basis for stepwise refinement and the programming calculus. *Science of Computer programming* 9, 3 (1987), 287–306.
- [34] Nathaniel Pinckney, Christopher Batten, Mingjie Liu, Haoxing Ren, and Brucek Khailany. 2025. Revisiting verilogval: A year of improvements in large-language models for hardware code generation. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 1–20.
- [35] Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Brucek Khailany, and Haoxing Ren. 2025. Comprehensive Verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on rtl design and verification. *arXiv preprint arXiv:2506.14074* (2025).
- [36] Klaus Pohl. 1996. *Requirements engineering: An overview*. RWTH, Fachgruppe Informatik Aachen.
- [37] Viorel Preoteasa and Stavros Tripakis. 2014. Refinement calculus of reactive systems. In *Proceedings of the 14th International Conference on Embedded Software*. 1–10.
- [38] Farzaneh Rabiei, Mark Zakharov, and Jose Renau. 2025. Beyond Verilog: Agents for Emerging HDLs. In *IEEE International Conference on Omni-layer Intelligent Systems (COINS)*.
- [39] Bernhard Rumpe and Cornel Klein. 1996. Automata describing object behavior. In *Object-Oriented Behavioral Specifications*. Springer, 265–286.
- [40] Peter Scholz. 1998. A refinement calculus for statecharts. In *International Conference on Fundamental Approaches to Software Engineering*. Springer, 285–301.
- [41] Douglas J Smith. 1996. VHDL & Verilog compared & contrasted—plus modeled example written in VHDL, Verilog and C. In *DAC 1996*.
- [42] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2023. Autochip: Automating hdl generation using llm feedback. *arXiv preprint arXiv:2311.04887* (2023).
- [43] YunDa Tsai, Mingjie Liu, and Haoxing Ren. 2024. Rtlfixer: Automatically fixing rtl syntax errors with large language model. In *DAC 2024*.
- [44] Yiting Wang, Guoheng Sun, Wanghao Ye, Gang Qu, and Ang Li. 2025. Verireason: Reinforcement learning with testbench feedback for reasoning-enhanced verilog generation. *arXiv preprint arXiv:2505.11849* (2025).
- [45] Niklaus Wirth. 1971. Program development by stepwise refinement. *Commun. ACM* 14, 4 (1971), 221–227.
- [46] Clifford Wolf, Johann Glaser, and Johannes Kepler. 2013. Yosys-a free verilog synthesis suite. In *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*, Vol. 97. 1–6.
- [47] Zhongzhi Yu, Mingjie Liu, Michael Zimmer, Yingyan Celine, Yong Liu, and Haoxing Ren. 2025. Spec2RTL-Agent: Automated Hardware Code Generation from Complex Specifications Using LLM Agent Systems. In *IEEE International Conference on LLM-Aided Design (ICLAD)*. 37–43.
- [48] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. 2025. Mage: A multi-agent engine for automated rtl code generation. In *DAC 2025*.
- [49] Yingjie Zhou, Renzhi Chen, Xinyu Li, Jingkai Wang, Zhigang Fang, Bowei Wang, Wenqiang Bai, Qilin Cao, and Lei Wang. 2025. VToT: Automatic Verilog Generation via LLMs with Tree of Thoughts Prompting. In *DATE 2025*.

8 Appendix A

In this appendix, we select 3 simple examples to illustrate the hardware refinement process.

8.1 Oscillator

The following example is an excerpt from [8]. It specifies an oscillator whose half-period is equal to the time frame.

```
- output Q
```

```
The module should emit a zero on Q in even
time frames, and a one on Q in odd time
frames.
```

It is straightforward to translate the above design specification into a formal specification. The output variable Q is supposed to be modified by the specification, so it is listed as the frame variable. By default, the time variable T is reset to 0 at initialization. Throughout the execution, the modulo operator is used to check the parity of the current time frame. Because the design never terminates and does not depend on any environment assumptions, the postcondition and the environment condition are false and true, respectively.

```
oscillator ==
  Q:[T = 0, Q = T % 2, false] true <--
```

We use an arrow to point out which specification statement is selected to be refined in the next step. The first design decision is to implement the above specification statement as a single process. It leverages the *Iteration* rule is to extract the process body from the specification. To apply this rule, the agent must choose an inductive invariant which holds at the start up of the loop and in between every two time frames. Here, the during condition itself is a valid inductive invariant.

```
⊢ "Iteration" with inv == (Q = T % 2)
process[output:Q] {
  Q:[T = t_0 ∧ Q = T % 2, Q = T % 2,
    T = (t_0 + 1) ∧ Q = T % 2] <--
}
```

The next design decision is to transform the process body into a single assignment statement. It can be realized by applying the *Non-Blocking Assignment* rule, which assigns the inverse of Q to Q and increments T by 1:

```
⊢ "Non-Blocking Assignment"
  Q <== ¬ Q;
```

At this point, all specifications have been transformed into concrete code, so the refinement is completed. The following implementation can be translated into a synthesizable RTL program.

```

oscillator:
process[output:Q] {
  Q <== ¬ Q;
}

```

8.2 Euclid's Algorithm 1

The following design specification is excerpted from [41]. It describes Euclid's algorithm, which computes the greatest common divisor of two integers.

- output A
- output B

The module computes the greatest common divisor between two positive numbers stored in registers A and B. It continually subtracts B from A while A is greater than B, or A from B while B is greater than A. This process continues until A and B are equal.

This specification gives a detailed description of the intended behavior in every time frame. An LTL formula can capture the transition relation of the system with the next time operator **X** and the previous time operator **Y**:

```

euclid1 == A,B:[A > 0 ∧ B > 0,
  A > B ∧ X(A = Y A - Y B ∧ B = Y B) ∨
  B > A ∧ X(B = Y B - Y A ∧ A = Y A) ∨
  A = B ∧ X(B = Y B ∧ A = Y A),
  false] true <--

```

Suppose that the agent decides to decompose the above process into two concurrent processes. After applying the *Bidirectional Composition* rule, the during condition of process *proc_b* becomes the environment condition of *proc_a*, and vice versa.

```

⊢ "Bidirectional Composition" with euclid
  == proc_a || proc_b
proc_a == A:[A > 0 ∧ B > 0,
  A > B ∧ X(A = Y A - Y B) ∨
  A <= B ∧ X(A = Y A),false] dur_b <--
proc_b == B:[A > 0 ∧ B > 0,
  B > A ∧ X(B = Y B - Y A) ∨
  B <= A ∧ X(B = Y B),false] dur_a

```

In the next step, the specification statement corresponding to *proc_a* is selected for further refinement. According to the *Iteration* rule, the during condition and the environment condition are merged to form the context of the process body. A predicate that is established at the start up of the process is chosen as the inductive invariant.

```

⊢ "Iteration" with inv == (A > 0 ∧ B > 0)
process[output:A] {

```

```

  A:[T = t_0 ∧ (A > 0 ∧ B > 0),
  (B > A ∧ X(B = Y B - Y A) ∨
  B <= A ∧ X(B = Y B)) ∧
  (A > B ∧ X(A = Y A - Y B) ∨
  A <= B ∧ X(A = Y A)),
  T = (t_0 + 1) ∧ A > 0 ∧ B > 0] <--
}

```

The above process body has two main drawbacks. First, the main functional requirements are encoded in the during condition, which deviates from the convention that uses pre-condition and postcondition to describe a loop body. Second, it contains the temporal operators **X** and **Y**, which are difficult to reason about. The *Unfolding* rewriting rule proposed by Manna and Pnueli [29] is applied to address these issues. To improve readability, in the postcondition of a process body, we use V_0 to denote the value of variable V at t_0 .

```

⊢ by Manna-Pnueli "Unfolding" and the
  definition of X
A:[T = t_0 ∧ (A > 0 ∧ B > 0), true,
  (B0 > A0 ∧ B = B0 - A0 ∨
  B0 <= A0 ∧ B = B0) ∧
  (A0 > B0 ∧ A = A0 - B0 ∨
  A0 <= B0 ∧ A = A0) ∧
  T = (t_0 + 1) ∧ A > 0 ∧ B > 0] <--

```

Afterward, the *Alternation* rule is used to decompose the specification statement into multiple guarded commands. Because only A is in the frame, the guards can be inferred directly from the clauses associated with A:

```

⊢ "Alternation" with G1 == (A > B) and G2
  == (A <= B)
if (A > B) then
  A:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧ A > B,
  true,
  T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
  (B0 > A0 ∧ B = B0 - A0 ∨
  B0 <= A0 ∧ B = B0) ∧
  (A0 > B0 ∧ A = A0 - B0 ∨
  A0 <= B0 ∧ A = A0)] <--
else if (A <= B) then
  A:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧ A <= B,
  true,
  T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
  (B0 > A0 ∧ B = B0 - A0 ∨
  B0 <= A0 ∧ B = B0) ∧
  (A0 > B0 ∧ A = A0 - B0 ∨
  A0 <= B0 ∧ A = A0)]

```

Suppose that the first branch is selected for further refinement in the next step. The *Strengthen Postcondition* rule is used to remove redundant information in the postcondition:

```

⊢ "Strengthen Postcondition"
A:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧ A > B,

```

```

true, T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
A0 > B0 ∧ B = B0 ∧ A = A0 - B0] <--

```

The above specification statement can be realized by a single non-blocking assignment statement:

```

⊆ "Non-Blocking Assignment"
A <== A - B; <--

```

The same refinement techniques can be applied to the *else if* branch as well as *proc_b*. The final code is displayed as follows:

```

euclid1:
process[output:A] {
  if A > B then A <== A - B;
  else skip;
}
process[output:B] {
  if B > A then B <== B - A;
  else skip;
}

```

8.3 Euclid's Algorithm 2

A specification may not explicitly define low-level hardware behavior. Instead, it may provide only abstract functional requirements:

- output A
- output B

The module computes the greatest common divisor between two positive numbers stored in registers A and B. Throughout this process, the greatest common divisor of A and B is always identical to the greatest common divisor of their initial values.

We use V_{init} to denote the value of variable V in the reset state. The functional requirements are directly mapped to the following specification statement:

```

euclid2 == A,B:[A > 0 ∧ B > 0,
gcd(A,B) = gcd(Ainit,Binit),
false] true <--

```

The *Iteration* rule is applied to extract the process body from the entire process:

```

⊆ "Iteration" with inv == (A > 0 ∧ B > 0)
∧ gcd(A,B) = gcd(Ainit,Binit)
process[output:A,B] {
  A,B:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(Ainit,Binit),
gcd(A,B) = gcd(Ainit,Binit),
T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(Ainit,Binit)] <--

```

```

}

```

To simplify reasoning for subsequent refinement steps, the agent weakens the unnecessary during condition to true:

```

⊆ "Weaken Context"
A,B:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(Ainit,Binit),true,
T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(Ainit,Binit)] <--

```

In the postcondition, A_{init} and B_{init} can be replaced by $A0$ and $B0$. Their equivalence is justified by the precondition.

```

⊆ "Rewriting" by the transitive property of
equality
A,B:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(Ainit,Binit),true,
T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(A0,B0)] <--

```

The agent then decides to decompose the specification statement into three branches. Each branch may lead to a different implementation.

```

⊆ "Alternation" with G1 == (A > B), G2 == (
A < B), and G3 == (A = B)
if (A > B) then
  A,B:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(Ainit,Binit) ∧ A > B,true,
T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(A0,B0)] <--
else if (A < B) then
  A,B:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(Ainit,Binit) ∧ A < B,true,
T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(A0,B0)]
else if (A = B) then
  A,B:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(Ainit,Binit) ∧ A = B,true,
T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(A0,B0)]

```

Suppose that the first branch is selected for further refinement in the next step. To ensure the progress of the process body, a ranking function must monotonically decrease in every iteration. This additional clause is conjoined to the postcondition with the *Strengthen Postcondition* rule.

```

⊆ "Strengthen Postcondition" with ranking
function A + B
A,B:[T = t_0 ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(Ainit,Binit),true,
T = (t_0 + 1) ∧ (A > 0 ∧ B > 0) ∧
gcd(A,B) = gcd(A0,B0)
∧ (A + B < A0 + B0)] <--

```

The above specification statement can be realized by a multi-assignment non-blocking assignment statement:

```
⊆ "Non-Blocking Assignment"  
A,B <== A - B,B; <--
```

After applying the same refinement techniques to the other two branches, we obtain the following code:

```
euclid2:  
process[output:A,B] {  
    if A > B then A,B <== A - B,B;  
    else if A < B then A,B <== A,B - A;  
    else skip;  
}
```